L3 Info Analyse Syntaxique

TD 12 (appliqué) – Analyse LL1 de la grammaire des expressions

Exercice E12.1: Analyse LL1 de la grammaire des expressions

On souhaite analyser/interpréter les expressions arithmétiques usuelles, telles qu'utilisées par exemple en langage Java. Pour simplifier notre propos, dans un premier temps, nous nous restreindrons aux opérateurs additifs et multiplicatifs.

On donne la grammaire suivante, de source E :

$$\begin{array}{lcl} E \to E + T & | & T \\ T \to T * F & | & F \\ F \to x & | & a \quad | \quad (E) \end{array}$$

Dans ce langage (et dans les activités pratiques qui suivront) le terminal a pourra être interprété comme une valeur immédiate et x comme une variable.

a. Dites pourquoi cette grammaire ne peut pas être LL1.

b. Calculez une grammaire équivalente sans récursivité à gauche.

c. Calculez la table des *P*remiers et des *S*uivants.

d. Calculez la table LL1. Cette grammaire est-elle LL1 ?

Exercice E12.2: Analyse LL1 de la grammaire des affectations

On se donne maintenant la grammaire des affectations, de source A, qui reprend la grammaire précédente en l'enrichissant de l'opération d'affectation (=) telle que définie dans le langage java [1].

$$\begin{array}{lcl} A \to x = A & | & E \\ E \to E + T & | & T \\ T \to T * F & | & F \\ F \to x & | & a \quad | \quad (A) \end{array}$$

Nota : Cette grammaire ne diffère de la précédente que par les règles du non terminal A et par la règle $F \to (A)$ qui se substitue à la règle $F \to (E)$.

a. Donnez une grammaire équivalente sans récursivité à gauche.

b. Calculez la table des *P*remiers et des *S*uivants.

d. Calculez la table LL1. Cette grammaire est-elle LL1 ?

e. Pensez vous que les conflits sans la table des *D*irecteurs peuvent être résolus facilement ? Essayez de proposer un moyen de rendre la table opérationnelle.

Exercice E12.3: Un programme d'analyse basé sur la méthode LL1

Nous souhaitons maintenant programmer une analyse LL1. Pour cela, la méthode actuelle suppose de programmer un automate à pile et d'utiliser la table des directeurs pour décider de la transition à suivre, en fonction du symbole non terminal en sommet de pile et du "début" du mot à analyser.

L'inconvénient de se programme est qu'il est peu lisible. La stratége que l'on retrouve leplus souvent d'appuie sur le fait que, dans un langage de programmation impératif, un appel à une fonction consiste à empiler le contexte de la fonction appelée. Un programme simple, contient donc une pile implicite : la pile des appels. Il suffit donc de s'arranger pour que chaque non terminal soit une action de votre programme. Ainsi une prédiction consistera à appeler la séquence des actions associées à la partie droite (lecture pour les terminaux, appel pour les non terminaux).

Pour illustrer ce principe nous utiliserons la grammaire, générant le langage $a^n b^n$, suivante :

$$S \to aSb \quad | \quad \varepsilon$$

a. Calculez les *Premiers*, *Suivants* et table LL1 de cette grammaire, est-elle LL1 ?

b. Construire un programme d'analyse en descente récursive à partir de cette grammaire.

1. Certains langages utilisent (à raison) d'autres opérateurs pour cette opération (ex : := en ALGOL ou en Pascal ou <- en OCaml, etc.)

[Construire une procédure S sans argument, qui s'appuie sur la table LL1 construite pour décider que faire, en fonction du caractère courant du mot.]

c. En utilisant le même principe, écrire le programme d'analyse en descente récursive pour la grammaire des expressions vue à l'exercice E12.1.